# The structures of Riktig

## Every class in theories/Structure/ – carriers, operations with their types, and laws – as Coq states them, generated by scripts/structures-md.py

**Magma** (M)
ops: `add (+) : M → M → M`
- `equivalence` : `Equivalence equ`
- `op_proper` : `Proper (equ ==> equ ==> equ) add`

**Semigroup** (S) extends `Magma`
ops: `add (+) : S → S → S`
- `is_associative` : `∀ a b x : S, a + b + x == a + (b + x)`

**Monoid** (M) extends `Semigroup`
ops: `zero (0) : M`, `add (+) : M → M → M`
- `is_left_identity` : `∀ x : M, 0 + x == x`
- `is_right_identity` : `∀ x : M, x + 0 == x`

**Group** (G) extends `Monoid`
ops: `zero (0) : G`, `add (+) : G → G → G`, `neg (- x) : G → G`
- `neg_proper` : `Proper (equ ==> equ) neg`
- `is_left_inverse` : `∀ a : G, - a + a == 0`
- `is_right_inverse` : `∀ a : G, a - a == 0`

**AbGroup** (A) extends `Group`
ops: `zero (0) : A`, `add (+) : A → A → A`, `neg (- x) : A → A`
- `is_commutative` : `∀ a b : A, a + b == b + a`

**Semiring** (R) extends `Monoid` (add_monoid), `Monoid` (mul_monoid)
ops: `zero (0) : R`, `add (+) : R → R → R`, `one (1) : R`, `mult (*) : R → R → R`
- `add_commutative` : `∀ a b : R, a + b == b + a`
- `is_left_distributive` : `∀ a x y : R, a * (x + y) == a * x + a * y`
- `is_right_distributive` : `∀ x y b : R, (x + y) * b == x * b + y * b`
- `mul_zero_left` : `∀ x : R, 0 * x == 0`
- `mul_zero_right` : `∀ x : R, x * 0 == 0`

**CommSemiring** (R) extends `Semiring`
ops: `zero (0) : R`, `add (+) : R → R → R`, `one (1) : R`, `mult (*) : R → R → R`
- `mul_commutative` : `∀ a b : R, a * b == b * a`

**Peano** (N)
ops: `zero (0) : N`, `succ : N → N`
- `equivalence` : `Equivalence equ`
- `succ_proper` : `Proper (equ ==> equ) succ`
- `succ_injective` : `∀ a b : N, succ a == succ b → a == b`
- `succ_not_zero` : `∀ n : N, succ n =/= 0`
- `nat_induction` : `∀ P : N → Prop, Proper (equ ==> iff) P → P 0 → (∀ n : N, P n → P (succ n)) → ∀ n : N, P n`

**PeanoArith** (N) extends `Peano`
ops: `zero (0) : N`, `succ : N → N`, `one (1) : N`, `add (+) : N → N → N`, `mult (*) : N → N → N`
- `add_proper` : `Proper (equ ==> equ ==> equ) add`
- `mul_proper` : `Proper (equ ==> equ ==> equ) mult`
- `one_succ` : `(1 : N) == succ 0`
- `add_zero` : `∀ a : N, a + 0 == a`
- `add_succ` : `∀ a b : N, a + succ b == succ (a + b)`
- `mul_zero` : `∀ a : N, a * 0 == 0`
- `mul_succ` : `∀ a b : N, a * succ b == a * b + a`

**Naturals** (N) extends `Peano`, `CommSemiring`
ops: `zero (0) : N`, `succ : N → N`, `one (1) : N`, `add (+) : N → N → N`, `mult (*) : N → N → N`
- `succ_add` : `∀ n : N, succ n == n + 1`

**NatOrder** (N) over `Naturals N`
ops: `le (<=) : N → N → Prop`
- `le_proper` : `Proper (equ ==> equ ==> iff) le`
- `le_add` : `∀ a b : N, a <= b ↔ exists c : N, a + c == b`

**NatRec** (N) over `Peano N`
ops: `natrec : ∀ A : Type, A → (N → A → A) → N → A`
- `natrec_proper` : `∀ (A : Type) (eA : EqRel A) (EA : Equivalence eA) (a : A) (f : N → A → A), Proper (equ ==> equ ==> equ) f → Proper (equ ==> equ) (natrec A a f)`
- `natrec_zero` : `∀ (A : Type) (eA : EqRel A) (EA : Equivalence eA) (a : A) (f : N → A → A), natrec A a f 0 == a`
- `natrec_succ` : `∀ (A : Type) (eA : EqRel A) (EA : Equivalence eA) (a : A) (f : N → A → A) (n : N), natrec A a f (succ n) == f n (natrec A a f n)`

**Finite** (I N) over `NatOrder N`
ops: `card : N`, `index : I → N`, `elem : N → I`
- `equivalence` : `Equivalence equ`
- `index_proper` : `Proper (equ ==> equ) index`
- `elem_proper` : `Proper (equ ==> equ) elem`
- `index_lt` : `∀ i : I, index i < card`
- `elem_index` : `∀ i : I, elem (index i) == i`
- `index_elem` : `∀ k : N, k < card → index (elem k) == k`

**Table** (I A T)
ops: `get : T → I → A`, `tabulate : (I → A) → T`
- `equivalence` : `Equivalence equ`
- `get_proper` : `Proper (equ ==> equ ==> equ) get`
- `tabulate_proper` : `Proper (pointwise_relation I equ ==> equ) tabulate`
- `get_tabulate` : `∀ (f : I → A) (i : I), get (tabulate f) i == f i`
- `tabulate_get` : `∀ t : T, tabulate (get t) == t`

**Equiv** (A B)
ops: `to : A → B`, `from : B → A`
- `to_proper` : `Proper (equ ==> equ) to`
- `from_proper` : `Proper (equ ==> equ) from`
- `from_to` : `∀ a : A, from (to a) == a`
- `to_from` : `∀ b : B, to (from b) == b`

**EquDec** (A)
- `equ_dec` : `∀ a b : A, Decidable (a == b)`

**LtDec** (A)
- `lt_dec` : `∀ a b : A, Decidable (a < b)`

**LeDec** (A)
- `le_dec` : `∀ a b : A, Decidable (a <= b)`

**BigOp** (I A T) over `Monoid A`, `Table I A T get tabulate`
ops: `bigop : T → A`
- `bigop_proper` : `Proper (equ ==> equ) bigop`
- `bigop_zero` : `bigop (tabulate (λ _ ⇒ 0)) == 0`
- `bigop_add` : `∀ s t : T, bigop (tabulate (λ i ⇒ get s i + get t i)) == bigop s + bigop t`
- `bigop_single` : `∀ (t : T) (i : I) (a : A), get t i == a → (∀ j : I, j =/= i → get t j == 0) → bigop t == a`

**Ring** (R) extends `AbGroup`, `Monoid`
ops: `zero (0) : R`, `add (+) : R → R → R`, `neg (- x) : R → R`, `one (1) : R`, `mult (*) : R → R → R`
- `is_left_distributive` : `∀ a x y : R, a * (x + y) == a * x + a * y`
- `is_right_distributive` : `∀ x y b : R, (x + y) * b == x * b + y * b`

**RingHom** (A B)
ops: `h : A → B`
- `hom_proper` : `Proper (equ ==> equ) h`
- `hom_additive` : `∀ x y : A, h (x + y) == h x + h y`
- `hom_multiplicative` : `∀ x y : A, h (x * y) == h x * h y`
- `hom_one` : `h 1 == (1 : B)`

**Field** (F) extends `Ring`
ops: `zero (0) : F`, `add (+) : F → F → F`, `neg (- x) : F → F`, `one (1) : F`, `mult (*) : F → F → F`, `inv : F → F`
- `inv_proper` : `Proper (equ ==> equ) inv`
- `is_mul_commutative` : `∀ a b : F, a * b == b * a`
- `is_mul_left_inverse` : `∀ a : F, nonzero F a → inv a * a == 1`
- `nontrivial` : `(1 : F) =/= 0`

**MinusOneNotASquare** (F) over `Field F`
- `two_squares_vanish` : `∀ a b : F, a * a + b * b == 0 → a == 0`

**ScaleLaws** (F V) over `Field F`
ops: `add (+) : V → V → V`, `scale (*:) : F → V → V`
- `scale_proper` : `Proper (equ ==> equ ==> equ) scale`
- `is_left_distributive` : `∀ (a : F) (x y : V), a *: (x + y) == a *: x + a *: y`
- `is_right_distributive` : `∀ (x y : F) (b : V), (x + y) *: b == x *: b + y *: b`
- `is_associative` : `∀ (a b : F) (x : V), (a * b) *: x == a *: (b *: x)`
- `is_left_identity` : `∀ x : V, 1 *: x == x`

**VectorSpace** (F V) over `Field F` extends `AbGroup`, `ScaleLaws`
ops: `zero (0) : V`, `add (+) : V → V → V`, `neg (- x) : V → V`, `scale (*:) : F → V → V`

**Linear** (F U V)
ops: `f : U → V`
- `proper` : `Proper (equ ==> equ) f`
- `is_additive` : `∀ x y : U, f (x + y) == f x + f y`
- `is_homogeneous` : `∀ (a : F) (x : U), f (a *: x) == a *: f x`

**InnerProductSpace** (F V) over `VectorSpace F V`
- `inner_proper` : `Proper (equ ==> equ ==> equ) (@inner F V _)`
- `inner_additive` : `∀ v : V, Additive V F add add (λ u ⇒ ⟪ u , v ⟫)`
- `inner_homogeneous` : `∀ v : V, Homogeneous F V F mult (λ a ⇒ a) (λ u ⇒ ⟪ u , v ⟫)`
- `is_commutative` : `∀ a b : V, ⟪ a, b ⟫ == ⟪ b, a ⟫`

**Integers** (Z) extends `Ring`
ops: `zero (0) : Z`, `add (+) : Z → Z → Z`, `neg (- x) : Z → Z`, `one (1) : Z`, `mult (*) : Z → Z → Z`
- `mul_commutative` : `∀ a b : Z, a * b == b * a`
- `int_induction` : `∀ P : Z → Prop, Proper (equ ==> iff) P → P 0 → (∀ z : Z, P z → P (z + 1)) → (∀ z : Z, P z → P (z + - (1 : Z))) → ∀ z : Z, P z`

**Rationals** (Q Z) over `Field Q`, `Integers Z` extends `RingHom`
ops: `cast : Z → Q`
- `cast_injective` : `∀ a b : Z, cast a == cast b → a == b`
- `is_fraction` : `∀ q : Q, exists a b : Z, cast b =/= 0 /\ q * cast b == cast a`

**OrderedField** (F) over `Field F`
ops: `le (<=) : F → F → Prop`, `abs : F → F`
- `le_proper` : `Proper (equ ==> equ ==> iff) le`
- `abs_proper` : `Proper (equ ==> equ) abs`
- `le_reflexive` : `Reflexive le`
- `le_transitive` : `Transitive le`
- `le_antisymmetric` : `Antisymmetric F equ le`
- `le_total` : `∀ a b : F, a <= b \/ b <= a`
- `le_stable` : `∀ a b : F, ~ ~ a <= b → a <= b`
- `add_le_mono_l` : `∀ a b c : F, a <= b → c + a <= c + b`
- `mul_nonneg` : `∀ a b : F, 0 <= a → 0 <= b → 0 <= a * b`
- `abs_of_nonneg` : `∀ a : F, 0 <= a → abs a == a`
- `abs_neg` : `∀ a : F, abs (- a) == abs a`

**Reals** (R) over `OrderedField R`
- `complete` : `∀ P : Pred R, (exists x : R, P x) → (exists b : R, ∀ x : R, P x → x <= b) → exists s : R, (∀ x : R, P x → x <= s) /\ (∀ b : R, (∀ x : R, P x → x <= b) → s <= b)`

**ApproxSqrt** (F) over `OrderedField F`
ops: `approx_sqrt : F → F → F`
- `approx_sqrt_nonneg` : `∀ x eps : F, 0 <= approx_sqrt x eps`
- `approx_sqrt_le` : `∀ x eps : F, 0 <= x → 0 < eps → approx_sqrt x eps * approx_sqrt x eps <= x`
- `approx_sqrt_close` : `∀ x eps : F, 0 <= x → 0 < eps → x <= (approx_sqrt x eps + eps) * (approx_sqrt x eps + eps)`

**ConstructiveReals** (R Q N) over `Ring R`, `Commutative R R mult`, `OrderedField Q`, `NatOrder N` extends `RingHom`
ops: `lt (<) : R → R → Set`, `inv_apart : ∀ x : R, lt x 0 + lt 0 x → R`, `abs : R → R`, `approx : R → Q → Q`, `cast : Q → R`
- `lt_asym` : `∀ x y : R, x < y → y < x → False`
- `lt_trans` : `∀ x y z : R, x < y → y < z → x < z`
- `lt_cotrans` : `∀ x y z : R, x < z → ((x < y) + (y < z))%type`
- `lt_equ_l` : `∀ x x' y : R, x == x' → x < y → x' < y`
- `lt_equ_r` : `∀ x y y' : R, y == y' → x < y → x < y'`
- `equ_of_le` : `∀ x y : R, x ⩽ y → y ⩽ x → x == y`
- `cast_lt` : `∀ q r : Q, q < r → cast q < cast r`
- `lt_cast` : `∀ q r : Q, cast q < cast r → q < r`
- `zero_lt_one` : `(0 : R) < 1`
- `add_lt_mono_l` : `∀ x y z : R, y < z → x + y < x + z`
- `add_lt_reg_l` : `∀ x y z : R, x + y < x + z → y < z`
- `mul_pos` : `∀ x y : R, 0 < x → 0 < y → 0 < x * y`
- `mul_le_mono_l` : `∀ c x y : R, 0 ⩽ c → x ⩽ y → c * x ⩽ c * y`
- `inv_l` : `∀ (x : R) (Hx : ((x < 0) + (0 < x))%type), inv_apart x Hx * x == 1`
- `inv_pos` : `∀ (x : R) (Hx : ((x < 0) + (0 < x))%type), 0 < x → 0 < inv_apart x Hx`
- `dense` : `∀ x y : R, x < y → {q : Q & ((x < cast q) * (cast q < y))%type}`
- `archimedean` : `∀ x : R, {q : Q & x < cast q}`
- `abs_proper` : `Proper (equ ==> equ) abs`
- `abs_le_iff` : `∀ x y : R, (x ⩽ y /\ - x ⩽ y) ↔ abs x ⩽ y`
- `abs_mul` : `∀ x y : R, abs (x * y) == abs x * abs y`
- `approx_close` : `∀ (x : R) (eps : Q), 0 < eps → abs (x + - cast (approx x eps)) ⩽ cast eps`
- `complete` : `∀ u : N → R, (∀ eps : Q, 0 < eps → {n : N & ∀ i j : N, n <= i → n <= j → abs (u i + - u j) ⩽ cast eps}) → {l : R & ∀ eps : Q, 0 < eps → {n : N & ∀ i : N, n <= i → abs (u i + - l) ⩽ cast eps}}`

**Trig** (R) over `Ring R`, `Commutative R R mult`
ops: `sin : R → R`, `cos : R → R`
- `sin_proper` : `Proper (equ ==> equ) sin`
- `cos_proper` : `Proper (equ ==> equ) cos`
- `sin_zero` : `sin 0 == 0`
- `cos_zero` : `cos 0 == 1`
- `sin_add` : `∀ a b : R, sin (a + b) == sin a * cos b + cos a * sin b`
- `cos_add` : `∀ a b : R, cos (a + b) == cos a * cos b + - (sin a * sin b)`
- `sin_neg` : `∀ a : R, sin (- a) == - sin a`
- `cos_neg` : `∀ a : R, cos (- a) == cos a`

**Category** (O)
ops: `cat_id : ∀ x : O, x ~> x`, `comp (∘) : ∀ x y z : O, y ~> z → x ~> y → x ~> z`
- `hom_equivalence` : `∀ x y : O, Equivalence (@equ (x ~> y) (hom_equ x y))`

- comp_proper : ∀ x y z : O, Proper (equ ==> equu ==> equ) (@comp O hom comp x y z)
- is_left_identity : ∀ x y : O, LeftIdentity (y ~> y) (x ~> y) (@comp O hom comp x y y) cat_id
- is_right_identity : ∀ x y : O, RightIdentity (x ~> x) (x ~> y) (@comp O hom comp x x y) cat_id
- comp_assoc : ∀ (w x y z : O) (f : y ~> z) (g : x ~> y) (h : w ~> x), f ∘ (g ∘ h) == (f ∘ g) ∘ h

**Functor** (C D) over Category C, Category D
ops: fmap : ∀ x y : C, x ~> y → F x ~> F y, F : C → D
- fmap_proper : ∀ x y : C, Proper (equ ==> equ) (@fmap C D _ _ F fmap x y)
- fmap_id : ∀ x : C, fmap (F := F) (cat_id (x := x)) == cat_id
- fmap_comp : ∀ (x y z : C) (f : y ~> z) (g : x ~> y), fmap (F := F) (f ∘ g) == fmap (F := F) f ∘ fmap (F := F) g

**NaturalTransformation** (C D) over Category C, Category D extends Functor (source), Functor (target)
ops: F G : C → D, eta : ∀ x : C, F x ~> G x
- naturality : ∀ (x y : C) (f : x ~> y), fmap (F := G) f ∘ eta x == eta y ∘ fmap (F := F) f

**Terminal** (O) over Category O
ops: term (1) : O, to_term : ∀ x : O, x ~> 1%object
- to_term_unique : ∀ (x : O) (f : x ~> term), f == to_term x

**Initial** (O) over Category O
ops: init (0) : O, from_init : ∀ x : O, 0%object ~> x
- from_init_unique : ∀ (x : O) (f : init ~> x), f == from_init x

**Products** (O) over Category O
ops: prod_obj (×) : O → O → O, pi1 : ∀ a b : O, (a * b)%object ~> a, pi2 : ∀ a b : O, (a * b)%object ~> b, lift (f ×× g) : ∀ x a b : O, x ~> a → x ~> b → x ~> (a * b)%object
- lift_proper : ∀ x a b : O, Proper (equ ==> equ ==> equ) (@lift O _ _ lift x a b)
- pi1_lift : ∀ (x a b : O) (f : x ~> a) (g : x ~> b), pi1 ∘ lift f g == f
- pi2_lift : ∀ (x a b : O) (f : x ~> a) (g : x ~> b), pi2 ∘ lift f g == g
- lift_unique : ∀ (x a b : O) (f : x ~> a) (g : x ~> b) (h : x ~> a × b), pi1 ∘ h == f → pi2 ∘ h == g → h == lift f g

**Coproducts** (O) over Category O
ops: coprod_obj (⨿) : O → O → O, inj1 : ∀ a b : O, a ~> (a + b)%object, inj2 : ∀ a b : O, b ~> (a + b)%object, desc (f ⨿⨿ g) : ∀ x a b : O, a ~> x → b ~> x → (a + b)%object ~> x
- desc_proper : ∀ x a b : O, Proper (equ ==> equ ==> equ) (@desc O _ _ desc x a b)
- desc_inj1 : ∀ (x a b : O) (f : a ~> x) (g : b ~> x), desc f g ∘ inj1 == f
- desc_inj2 : ∀ (x a b : O) (f : a ~> x) (g : b ~> x), desc f g ∘ inj2 == g
- desc_unique : ∀ (x a b : O) (f : a ~> x) (g : b ~> x) (h : a ⨿ b ~> x), h ∘ inj1 == f → h ∘ inj2 == g → h == desc f g

**Bicartesian** (O) over Category O extends Terminal, Initial, Products, Coproducts
ops: term (1) : O, to_term : ∀ x : O, x ~> 1%object, init (0) : O, from_init : ∀ x : O, 0%object ~> x, prod_obj (×) : O → O → O, pi1 : ∀ a b : O, (a * b)%object ~> a, pi2 : ∀ a b : O, (a * b)%object ~> b, lift (f ×× g) : ∀ x a b : O, x ~> a → x ~> b → x ~> (a * b)%object, coprod_obj (⨿) : O → O → O, inj1 : ∀ a b : O, a ~> (a + b)%object, inj2 : ∀ a b : O, b ~> (a + b)%object, desc (f ⨿⨿ g) : ∀ x a b : O, a ~> x → b ~> x → (a + b)%object ~> x

**InitialAlgebra** (O) over Category O extends Functor
ops: build : F t ~> t, cata : ∀ x : O, F x ~> x → t ~> x, F : O → O, t : O
- cata_proper : ∀ x : O, Proper (equ ==> equ) (cata (F := F) (t := t) x)
- cata_build : ∀ (x : O) (h : F x ~> x), cata x h ∘ build == h ∘ fmap (cata x h)
- cata_unique : ∀ (x : O) (h : F x ~> x) (u : t ~> x), u ∘ build == h ∘ fmap u → u == cata x h

**Exponentials** (O) over Products O
ops: exp_obj (⟹) : O → O → O, eval : ∀ a b : O, ((a ⟹ b) * a)%object ~> b, curry : ∀ x a b : O, (x * a)%object ~> b → x ~> a ⟹ b
- curry_proper : ∀ x a b : O, Proper (equ ==> equ) (@curry O _ _ _ curry x a b)
- eval_curry : ∀ (x a b : O) (h : x × a ~> b), eval ∘ lift (curry h ∘ pi1) pi2 == h
- curry_unique : ∀ (x a b : O) (h : x × a ~> b) (k : x ~> a ⟹ b), eval ∘ lift (k ∘ pi1) pi2 == h → k == curry h

**CartesianClosed** (O) over Category O extends Terminal, Products, Exponentials
ops: term (1) : O, to_term : ∀ x : O, x ~> 1%object, prod_obj (×) : O → O → O, pi1 : ∀ a b : O, (a * b)%object ~> a, pi2 : ∀ a b : O, (a * b)%object ~> b, lift (f ×× g) : ∀ x a b : O, x ~> a → x ~> b → x ~> (a * b)%object, exp_obj (⟹) : O → O → O, eval : ∀ a b : O, ((a ⟹ b) * a)%object ~> b, curry : ∀ x a b : O, (x * a)%object ~> b → x ~> a ⟹ b

**Iso** (O) over Category O
ops: a b : O, to : a ~> b, from : b ~> a
- hom_inv_id : from ∘ to == cat_id
- inv_hom_id : to ∘ from == cat_id

**Channel** (O) over Category O
ops: A W : O, encode : A ~> W, decode : W ~> A
- decode_encode : decode ∘ encode == cat_id

**NormedSpace** (F V) over VectorSpace F V
ops: norm (‖ v ‖) : V → F
- norm_proper : Proper (equ ==> equ) (@norm F V norm)
- norm_nonneg : ∀ v : V, 0 <= ‖ v ‖

- eq_zero_of_norm_eq_zero : ∀ v : V, ‖ v ‖ == 0 → v == 0
- norm_smul : ∀ (a : F) (v : V), ‖ a *: v ‖ == abs a * ‖ v ‖
- norm_add_le : ∀ u v : V, ‖ u + v ‖ <= ‖ u ‖ + ‖ v ‖

**Algebra** (F A) over Field F extends VectorSpace, Ring
ops: zero (0) : A, add (+) : A → A → A, neg (- x) : A → A, one (1) : A, mult (*) : A → A → A, scale (*:) : F → A → A
- smul_mul_assoc : ∀ (a : F) (x y : A), (a *: x) * y == a *: (x * y)
- mul_smul_comm : ∀ (a : F) (x y : A), x * (a *: y) == a *: (x * y)

**GeometricAlgebra** (F V A) over InnerProductSpace F V extends Linear
ops: vec : V → A
- vec_sq : ∀ u : V, vec u * vec u == ⟪ u , u ⟫ *: 1

**GradeInvolution** (F V A) over GeometricAlgebra F V A
ops: involute : A → A
- involute_proper : Proper (equ ==> equ) (@involute A involute)
- involute_additive : ∀ x y : A, involute (x + y) == involute x + involute y
- involute_multiplicative : ∀ x y : A, involute (x * y) == involute x * involute y
- involute_homogeneous : ∀ (a : F) (x : A), involute (a *: x) == a *: involute x
- involute_one : involute (1 : A) == 1
- involute_involute : ∀ x : A, involute (involute x) == x
- involute_vec : ∀ u : V, involute (vec u) == - vec u

**Limits** (F N V) over VectorSpace F V, NatOrder N
ops: tendsto : (N → V) → V → Type, cauchy : (N → V) → Type
- tendsto_proper : ∀ (u u' : N → V) (l l' : V), (∀ n : N, u n == u' n) → l == l' → tendsto u l → tendsto u' l'
- tendsto_unique : ∀ (u : N → V) (l l' : V), tendsto u l → tendsto u l' → l == l'
- tendsto_const : ∀ c : V, tendsto (λ _ ⇒ c) c
- tendsto_add : ∀ (u v : N → V) (l m : V), tendsto u l → tendsto v m → tendsto (λ n ⇒ u n + v n) (l + m)
- tendsto_neg : ∀ (u : N → V) (l : V), tendsto u l → tendsto (λ n ⇒ - u n) (- l)
- tendsto_smul : ∀ (a : F) (u : N → V) (l : V), tendsto u l → tendsto (λ n ⇒ a *: u n) (a *: l)
- tendsto_tail : ∀ (u : N → V) (l : V), tendsto u l → tendsto (λ n ⇒ u (succ n)) l
- cauchy_proper : ∀ u u' : N → V, (∀ n : N, u n == u' n) → cauchy u → cauchy u'
- cauchy_of_tendsto : ∀ (u : N → V) (l : V), tendsto u l → cauchy u
- cauchy_add : ∀ u v : N → V, cauchy u → cauchy v → cauchy (λ n ⇒ u n + v n)
- cauchy_neg : ∀ u : N → V, cauchy u → cauchy (λ n ⇒ - u n)
- cauchy_smul : ∀ (a : F) (u : N → V), cauchy u → cauchy (λ n ⇒ a *: u n)
- cauchy_tail : ∀ u : N → V, cauchy u → cauchy (λ n ⇒ u (succ n))

**Complete** (F N V) over Limits F N V
ops: lim : ∀ u : N → V, cauchy0 u → V
- lim_tendsto : ∀ (u : N → V) (Hu : cauchy u), tendsto u (lim u Hu)

**Archimedean** (F N) over OrderedField F, NatOrder N
ops: of_nat : N → F
- of_nat_proper : Proper (equ ==> equ) of_nat
- of_nat_zero : of_nat 0 == 0
- of_nat_succ : ∀ n : N, of_nat (succ n) == of_nat n + 1
- archimedean : ∀ q : F, {n : N & q < of_nat n}

**Complex** (F C) over Algebra F C, Commutative C C mult
ops: imaginary : C, re : C → F, im : C → F
- i_sq : (imaginary : C) * imaginary == - (1 : C)
- re_proper : Proper (equ ==> equ) (@re F C re)
- im_proper : Proper (equ ==> equ) (@im F C im)
- re_additive : ∀ x y : C, re (x + y) == re x + re y
- im_additive : ∀ x y : C, im (x + y) == im x + im y
- re_homogeneous : ∀ (a : F) (x : C), re (a *: x) == a * re x
- im_homogeneous : ∀ (a : F) (x : C), im (a *: x) == a * im x
- re_one : re (1 : C) == (1 : F)
- im_one : im (1 : C) == (0 : F)
- re_i : re (imaginary : C) == (0 : F)
- im_i : im (imaginary : C) == (1 : F)
- re_im : ∀ z : C, z == re z *: 1 + im z *: imaginary

**Frame** (F V A I N) over GradeInvolution F V A, Finite I N
ops: top : A → F, basis : I → V
- char_not_two : (1 + 1 : F) =/= 0
- basis_proper : Proper (equ ==> equ) basis
- top_proper : Proper (equ ==> equ) (@top F A top)
- top_additive : ∀ x y : A, top (x + y) == top x + top y
- top_homogeneous : ∀ (a : F) (x : A), top (a *: x) == a * top x
- top_pseudoscalar : top (natrec A 1 (λ k acc ⇒ inv (1 + 1) *: (acc * vec (basis (elem k)) + vec (basis (elem k)) * involute acc)) card) == 1

**Upd** (I V)
ops: upd : (I → V) → I → V → I → V

- `upd_at` : `∀ (vs : I → V) (i : I) (x : V) (j : I), j == i → upd vs i x j == x`
- `upd_off` : `∀ (vs : I → V) (i : I) (x : V) (j : I), j =/= i → upd vs i x j == vs j`
- `upd_proper` : `∀ (vs : I → V) (i : I) (x : V), Proper (equ ==> equu) vs → Proper (equ ==> equ) (upd vs i x)`

**Multilinear** (F I V) over `VectorSpace F V`
ops: `form : (I → V) → F`
- `form_proper` : `∀ vs vs' : I → V, Proper (equ ==> equ) vs → Proper (equ ==> equ) vs' → (∀ i : I, vs i == vs' i) → form vs == form vs'`
- `slot_additive` : `∀ (vs : I → V) (j : I), Proper (equ ==> equ) vs → Additive V F add add (λ x ⇒ form (upd vs j x))`
- `slot_homogeneous` : `∀ (vs : I → V) (j : I), Proper (equ ==> equ) vs → Homogeneous F V F mult (λ a ⇒ a) (λ x ⇒ form (upd vs j x))`

**Alternating** (F I V)
ops: `form : (I → V) → F`
- `alternating` : `∀ (vs : I → V) (i j : I), Proper (equ ==> equ) vs → i =/= j → vs i == vs j → form vs == 0`

**Determinant** (F I V) over `VectorSpace F V` extends `Multilinear`, `Alternating`
ops: `det : (I → V) → F`, `basis : I → V`
- `det_basis` : `det basis == 1`

**Sequence** (C S)
ops: `step : S → C → S → Prop`
- `elements` : `Equivalence (@equ C equ)`
- `equivalence` : `Equivalence (@equ S equ)`
- `step_proper` : `Proper (equ ==> equ ==> equ ==> iff) (@step C S step)`
- `step_deterministic` : `∀ (s : S) (c c' : C) (s' s'' : S), step s c s' → step s c' s'' → c == c' /\ s' == s''`

**Parser** (C S)
ops: `ret : ∀ A : Type, A → P A`, `bind (>>=) : ∀ A B : Type, P A → (A → P B) → P B`, `fail : ∀ A : Type, P A`, `alt (<|>) : ∀ A : Type, P A → P A → P A`, `item : P C`, `parses : ∀ A : Type, EqRel A → P A → S → A → S → Prop`, `P : Type → Type`
- `parser_equ` : `∀{Setoid A} (p q : P A), p == q ↔ (∀ (s s' : S) (a : A), parses p s a s' ↔ parses q s a s')`
- `parses_proper` : `∀{Setoid A} (p : P A), Proper (equ ==> equ ==> equ ==> iff) (parses p)`
- `parses_ret` : `∀{Setoid A} (a a' : A) (s s' : S), parses (ret a : P A) s a' s' ↔ a' == a / s' == s`
- `parses_bind` : `∀{Setoid A} {Setoid B} (p : P A) (f : A → P B) (s s'' : S) (b : B), Proper (equ ==> equ) f → (parses (p >>= f : P B) s b s'' ↔ exists (a : A) (s' : S), parses p s a s' /\ parses (f a) s' b s'')`
- `parses_fail` : `∀{Setoid A} (s s' : S) (a : A), ~ parses (fail : P A) s a s'`
- `parses_alt` : `∀{Setoid A} (p q : P A) (s s' : S) (a : A), parses (p <|> q) s a s' ↔ parses p s a s' / parses q s a s'`
- `parses_item` : `∀ (s s' : S) (c : C), parses (item : P C) s c s' ↔ step s c s'`

**CPSLambda** (N T Sb) over `LambdaCalculus N T Sb`
ops: `trivial : T → Prop`, `tail : T → Prop`, `cps : T → T`, `psi : T → T`, `evaluates : T → T → Prop`
- `trivial_var` : `∀ n : N, trivial (var n)`
- `trivial_lam` : `∀ b : T, tail b → trivial (lam b)`
- `tail_of_trivial` : `∀ v : T, trivial v → tail v`
- `tail_call` : `∀ f v : T, trivial f → trivial v → tail (app f v)`
- `tail_call2` : `∀ f v w : T, trivial f → trivial v → trivial w → tail (app (app f v) w)`
- `cps_trivial` : `∀ t : T, trivial (cps t)`
- `cps_value` : `∀ v k : T, trivial v → app (cps v) k == app k (psi v)`
- `cps_runs` : `∀ l v k : T, evaluates l v → app (cps l) k == app k (psi v)`

**Defunctionalization** (N T Sb K) over `CPSLambda N T Sb`
ops: `is_value : T → Prop`, `halt : K`, `karg : T → K → K`, `kfun : T → K → K`, `reify : K → T`, `druns : T → K → T → Prop`
- `value_lam` : `∀ b : T, is_value (lam b)`
- `value_evaluates` : `∀ v : T, is_value v → evaluates v v`
- `reify_halt` : `∀ t : T, app (reify halt) t == t`
- `reify_karg` : `∀ (f a : T) (k : K), app (cps (app f a)) (reify k) == app (cps f) (reify (karg a k))`
- `reify_kfun` : `∀ (a f : T) (k : K), is_value f → app (reify (karg a k)) (psi f) == app (cps a) (reify (kλ f k))`
- `reify_call` : `∀ (b v : T) (k : K), is_value v → app (reify (kλ (lam b) k)) (psi v) == app (cps (b.[v .: sub_id])) (reify k)`
- `druns_sound` : `∀ (t v : T) (k : K), druns t k v → app (cps t) (reify k) == psi v`
- `druns_complete` : `∀ l v : T, evaluates l v → druns l halt v`

**Graded** (N F A) over `Naturals N`, `Algebra F A`
ops: `grade_part (⟨ x ⟩_ k) : N → A → A`, `top_grade : N`
- `grade_linear` : `∀ k : N, Linear F A A (grade_part k)`
- `grade_index_proper` : `∀ (k k' : N) (x : A), k == k' → ⟨x⟩_k == ⟨x⟩_k'`
- `grade_idempotent` : `∀ (k : N) (x : A), ⟨⟨x⟩_k⟩_k == ⟨x⟩_k`
- `grade_orthogonal` : `∀ (k l : N) (x : A), k =/= l → ⟨⟨x⟩_l⟩_k == 0`
- `grade_complete` : `∀ x : A, natrec A 0 (λ k acc ⇒ acc + ⟨x⟩_k) (succ top_grade) == x`

**Reversal** (A) over `Ring A`
ops: `reverse : A → A`
- `reverse_proper` : `Proper (equ ==> equ) (@reverse A reverse)`
- `reverse_additive` : `∀ x y : A, reverse (x + y) == reverse x + reverse y`
- `reverse_anti` : `∀ x y : A, reverse (x * y) == reverse y * reverse x`
- `reverse_involutive` : `∀ x : A, reverse (reverse x) == x`

**GradedGA** (N F V A) over `GradeInvolution F V A`, `Naturals N` extends `Graded`, `Reversal`
ops: `grade_part (( x )_ k) : N → A → A`, `scalar_part : A → F`, `reverse : A → A`, `top_grade : N`
- `one_grade_zero : ⟨(1 : A)⟩_0 == 1`
- `grade_zero_scalar : ∀ x : A, ⟨x⟩_0 == scalar_part x *: 1`
- `vec_grade_one : ∀ u : V, ⟨vec u⟩_(succ 0) == vec u`
- `vec_mul_grade_zero : ∀ (u : V) (x : A), vec u * ⟨x⟩_0 == ⟨vec u * ⟨x⟩_0⟩_(succ 0)`
- `vec_mul_grade_succ : ∀ (u : V) (x : A) (k : N), vec u * ⟨x⟩_(succ k) == ⟨vec u * ⟨x⟩_(succ k)⟩_k + ⟨vec u * ⟨x⟩_(succ k)⟩_(succ (succ k))`
- `involute_grade : ∀ (k : N) (x : A), involute ⟨x⟩_k == ⟨involute x⟩_k`
- `reverse_grade : ∀ (k : N) (x : A), reverse ⟨x⟩_k == ⟨reverse x⟩_k`
- `reverse_homogeneous : ∀ (a : F) (x : A), reverse (a *: x) == a *: reverse x`
- `reverse_vec : ∀ u : V, reverse (vec u) == vec u`
- `reverse_involute : ∀ x : A, reverse (involute x) == involute (reverse x)`

**Rotor** (F V A) over `GradeInvolution F V A`
ops: `R : A`
- `rotor_even : involute R == R`
- `rotor_unit_r : R * reverse R == 1`
- `rotor_unit_l : reverse R * R == 1`

**Basis** (F I V N) over `VectorSpace F V`, `Finite I N`
ops: `coords : V → I → F`, `basis : I → V`
- `basis_proper : Proper (equ ==> equ) basis`
- `coords_proper : Proper (equ ==> equ ==> equ) (@coords F I V coords)`
- `coords_additive : ∀ i : I, Additive V F add add (λ v ⇒ coords v i)`
- `coords_homogeneous : ∀ i : I, Homogeneous F V F mult (λ a ⇒ a) (λ v ⇒ coords v i)`
- `reconstruction : ∀ v : V, natrec V 0 (λ k acc ⇒ acc + coords v (elem k) *: basis (elem k)) card == v`
- `coords_basis : ∀ i j : I, coords (basis j) i == (if EquDec.equ_dec i j then (1 : F) else 0)`

**LambdaCalculus** (N T Sb) over `Naturals N`
ops: `var : N → T`, `lam : T → T`, `app : T → T → T`, `substitute (t .[ s ]) : T → Sb → T`, `sub_id : Sb`, `sub_shift : Sb`, `sub_cons (.:) : T → Sb → Sb`, `sub_up : Sb → Sb`, `reduces (-->*) : T → T → Prop`
- `equ_join : ∀ t u : T, t == u ↔ (exists w : T, t -→* w /\ u -→* w)`
- `reduces_refl : ∀ t : T, t -→* t`
- `reduces_trans : ∀ t u v : T, t -→* u → u -→* v → t -→* v`
- `reduces_lam : ∀ b b' : T, b -→* b' → lam b -→* lam b'`
- `reduces_app : ∀ f f' a a' : T, f -→* f' → a -→* a' → app f a -→* app f' a'`
- `reduces_subst : ∀ (t t' : T) (s : Sb), t -→* t' → t.[s] -→* t'.[s]`
- `beta : ∀ b u : T, app (lam b) u -→* b.[u .: sub_id]`
- `var_injective : ∀ n m : N, (var n : T) == var m → n == m`
- `subst_id : ∀ t : T, t.[sub_id] == t`
- `var_zero_cons : ∀ (u : T) (s : Sb), (var 0 : T).[u .: s] == u`
- `var_succ_cons : ∀ (n : N) (u : T) (s : Sb), (var (succ n) : T).[u .: s] == (var n : T).[s]`
- `var_shift : ∀ n : N, (var n : T).[sub_shift] == var (succ n)`
- `var_zero_up : ∀ s : Sb, (var 0 : T).[sub_up s] == var 0`
- `var_succ_up : ∀ (n : N) (s : Sb), (var (succ n) : T).[sub_up s] == ((var n : T).[s]).[sub_shift]`
- `app_subst : ∀ (f a : T) (s : Sb), (app f a).[s] == app (f.[s]) (a.[s])`
- `lam_subst : ∀ (b : T) (s : Sb), (lam b).[s] == lam (b.[sub_up s])`
- `shift_cons : ∀ (t u : T) (s : Sb), (t.[sub_shift]).[u .: s] == t.[s]`
- `shift_up : ∀ (t : T) (s : Sb), (t.[sub_shift]).[sub_up s] == (t.[s]).[sub_shift]`
- `confluence : ∀ t u v : T, t -→* u → t -→* v → exists w : T, u -→* w /\ v -→* w`

**StlcType** (O K) over `Bicartesian O` extends `InitialAlgebra`
ops: `build : shape den b Ty ~> Ty`, `cata : ∀ x : O, shape den b x ~> x → Ty ~> x`, `den : K → O`, `b : K`, `Ty : O`

**Stlc** (O K) over `Bicartesian O` extends `InitialAlgebra`
ops: `build : shape den n ty T ~> T`, `cata : ∀ x : O, shape den n ty x ~> x → T ~> x`, `den : K → O`, `n ty : K`, `T : O`

**StlcCalculus** (N Ty T Sb) over `Naturals N`
ops: `var : N → T`, `tylam : Ty → T → T`, `app : T → T → T`, `substitute (t .[ s ]) : T → Sb → T`, `sub_id : Sb`, `sub_shift : Sb`, `sub_cons (.:) : T → Sb → Sb`, `sub_up : Sb → Sb`, `reduces (-->*) : T → T → T → Prop`
- `equ_join : ∀ t u : T, t == u ↔ (exists w : T, t -→* w /\ u -→* w)`
- `reduces_refl : ∀ t : T, t -→* t`
- `reduces_trans : ∀ t u v : T, t -→* u → u -→* v → t -→* v`
- `reduces_tylam : ∀ (A : Ty) (b b' : T), b -→* b' → tylam A b -→* tylam A b'`
- `reduces_app : ∀ f f' a a' : T, f -→* f' → a -→* a' → app f a -→* app f' a'`
- `reduces_subst : ∀ (t t' : T) (s : Sb), t -→* t' → t.[s] -→* t'.[s]`
- `tylam_proper : ∀ (A A' : Ty) (b : T), A == A' → tylam A b == tylam A' b`
- `beta : ∀ (A : Ty) (b u : T), app (tylam A b) u -→* b.[u .: sub_id]`
- `var_injective : ∀ n m : N, (var n : T) == var m → n == m`
- `subst_id : ∀ t : T, t.[sub_id] == t`
- `var_zero_cons : ∀ (u : T) (s : Sb), (var 0 : T).[u .: s] == u`
- `var_succ_cons : ∀ (n : N) (u : T) (s : Sb), (var (succ n) : T).[u .: s] == (var n : T).[s]`
- `var_shift : ∀ n : N, (var n : T).[sub_shift] == var (succ n)`

- var_zero_up : ∀ s : Sb, (var 0 : T).[sub_up s] == var 0
- var_succ_up : ∀ (n : N) (s : Sb), (var (succ n) : T).[sub_up s] == ((var n : T).[s]).[sub_shift]
- app_subst : ∀ (f a : T) (s : Sb), (app f a).[s] == app (f.[s]) (a.[s])
- tylam_subst : ∀ (A : Ty) (b : T) (s : Sb), (tylam A b).[s] == tylam A (b.[sub_up s])
- shift_cons : ∀ (t u : T) (s : Sb), (t.[sub_shift]).[u .: s] == t.[s]
- shift_up : ∀ (t : T) (s : Sb), (t.[sub_shift]).[sub_up s] == (t.[s]).[sub_shift]
- confluence : ∀ t u v : T, t -→* u → t -→* v → exists w : T, u -→* w /\ v -→* w

**StlcTyping** (N Ty T Sb G) over StlcCalculus N Ty T Sb, NatOrder N leN
ops: lookup_ : G → N → Ty, extend_ : Ty → G → G, empty_ : G, bound_ : G → N, typing : G → T → Ty → Prop
- typing_proper : ∀ (g : G) (t : T) (A A' : Ty), A == A' → typing g t A → typing g t A'
- typed_var : ∀ (g : G) (n : N) (A : Ty), succ n <= bound_ g → lookup_ g n == A → typing g (var n) A
- typed_tylam : ∀ (g : G) (A C : Ty) (b : T), typing (extend_ A g) b C → typing g (tylam A b) (A -→ C)
- typed_app : ∀ (g : G) (A C : Ty) (f a : T), typing g f (A -→ C) → typing g a A → typing g (app f a) C
- lookup_extend_zero : ∀ (g : G) (A : Ty), lookup_ (extend_ A g) 0 == A
- lookup_extend_succ : ∀ (g : G) (A : Ty) (n : N), lookup_ (extend_ A g) (succ n) == lookup_ g n
- bound_extend : ∀ (g : G) (A : Ty), bound_ (extend_ A g) == succ (bound_ g)
- bound_empty : bound_ empty_ == 0
- preservation : ∀ (g : G) (t t' : T) (A : Ty), typing g t A → t -→* t' → typing g t' A
- typed_reduces : ∀ (t : T) (A C : Ty), typing empty_ t (A -→ C) → exists (A' : Ty) (b : T), t -→* tylam A' b

**Signature** (K S)
ops: zero (0) : S, one (1) : S, self : S, const : K → S, add (+) : S → S → S, mult (*) : S → S → S, poly_rec : ∀ B : Type, B → B → B → (K → B) → (B → B → B) → (B → B → B) → S → B
- rec_zero : ∀ (B : Type) (z o s : B) (c : K → B) (p t : B → B → B), poly_rec B z o s c p t (0 : S) = z
- rec_one : ∀ (B : Type) (z o s : B) (c : K → B) (p t : B → B → B), poly_rec B z o s c p t (1 : S) = o
- rec_self : ∀ (B : Type) (z o s : B) (c : K → B) (p t : B → B → B), poly_rec B z o s c p t (self : S) = s
- rec_const : ∀ (B : Type) (z o s : B) (c : K → B) (p t : B → B → B) (k : K), poly_rec B z o s c p t (const k : S) = c k
- rec_plus : ∀ (B : Type) (z o s : B) (c : K → B) (p t : B → B → B) (a b : S), poly_rec B z o s c p t (a + b) = p (poly_rec B z o s c p t a) (poly_rec B z o s c p t b)
- rec_times : ∀ (B : Type) (z o s : B) (c : K → B) (p t : B → B → B) (a b : S), poly_rec B z o s c p t (a * b) = t (poly_rec B z o s c p t a) (poly_rec B z o s c p t b)
- rec_unique : ∀ (B : Type) (z o s : B) (c : K → B) (p t : B → B → B) (u : S → B), u 0 = z → u 1 = o → u self = s → (∀ k : K, u (const k) = c k) → (∀ a b : S, u (a + b) = p (u a) (u b)) → (∀ a b : S, u (a * b) = t (u a) (u b)) → ∀ q : S, u q = poly_rec B z o s c p t q

**ListRec** (X L)
ops: empty : L, prepend : X → L → L, list_rec : ∀ B : Type, B → (X → B → B) → L → B
- list_rec_proper : ∀ (B : Type) (eB : EqRel B) (EB : Equivalence eB) (n : B) (c : X → B → B), Proper (equ ==> equ ==> equ) c → Proper (equ ==> equ) (list_rec B n c : L → B)
- list_rec_empty : ∀ (B : Type) (eB : EqRel B) (EB : Equivalence eB) (n : B) (c : X → B → B), list_rec B n c (empty : L) == n
- list_rec_prepend : ∀ (B : Type) (eB : EqRel B) (EB : Equivalence eB) (n : B) (c : X → B → B) (x : X) (l : L), list_rec B n c (prepend x l) == c x (list_rec B n c l)

**TermRec** (N T)
ops: var : N → T, lam : T → T, app : T → T → T, termrec : ∀ B : Type, (N → B) → (B → B) → (B → B → B) → T → B
- termrec_proper : ∀ (B : Type) (eB : EqRel B) (EB : Equivalence eB) (v : N → B) (l : B → B) (a : B → B → B), Proper (equ ==> equ) v → Proper (equ ==> equ) l → Proper (equ ==> equ ==> equ) a → Proper (equ ==> equ) (termrec B v l a : T → B)
- termrec_var : ∀ (B : Type) (eB : EqRel B) (EB : Equivalence eB) (v : N → B) (l : B → B) (a : B → B → B) (n : N), termrec B v l a (var n : T) == v n
- termrec_lam : ∀ (B : Type) (eB : EqRel B) (EB : Equivalence eB) (v : N → B) (l : B → B) (a : B → B → B) (b : T), termrec B v l a (lam b) == l (termrec B v l a b)
- termrec_app : ∀ (B : Type) (eB : EqRel B) (EB : Equivalence eB) (v : N → B) (l : B → B) (a : B → B → B) (f u : T), termrec B v l a (app f u) == a (termrec B v l a f) (termrec B v l a u)

**TypedTermRec** (N Ty T)
ops: var : N → T, tylam : Ty → T → T, app : T → T → T, stermrec : ∀ B : Type, (N → B) → (Ty → B → B) → (B → B → B) → T → B
- stermrec_proper : ∀ (B : Type) (eB : EqRel B) (EB : Equivalence eB) (v : N → B) (l : Ty → B → B) (a : B → B → B), Proper (equ ==> equ) v → Proper (equ ==> equ ==> equ) l → Proper (equ ==> equ ==> equ) a → Proper (equ ==> equ) (stermrec B v l a : T → B)
- stermrec_var : ∀ (B : Type) (eB : EqRel B) (EB : Equivalence eB) (v : N → B) (l : Ty → B → B) (a : B → B → B) (n : N), stermrec B v l a (var n : T) == v n
- stermrec_tylam : ∀ (B : Type) (eB : EqRel B) (EB : Equivalence eB) (v : N → B) (l : Ty → B → B) (a : B → B → B) (A : Ty) (b : T), stermrec B v l a (tylam A b) == l A (stermrec B v l a b)
- stermrec_app : ∀ (B : Type) (eB : EqRel B) (EB : Equivalence eB) (v : N → B) (l : Ty → B → B) (a : B → B → B) (f u : T), stermrec B v l a (app f u) == a (stermrec B v l a f) (stermrec B v l a u)

**Derivative** (F U V) over VectorSpace F U
ops: differential : (U → V) → U → (U → V) → Prop
- differential_proper : ∀ (f f' : U → V) (a a' : U) (L L' : U → V), (∀ x : U, f x == f' x) → a == a' → (∀ h : U, L h == L' h) → differential f a L → differential f' a' L'

- `differential_linear` : `∀ (f : U → V) (a : U) (L : U → V), differential f a L → Linear F U V L`
- `differential_unique` : `∀ (f : U → V) (a : U) (L L' : U → V), differential f a L → differential f a L' → ∀ h : U, L h == L' h`
- `differential_const` : `∀ (v : V) (a : U), differential (λ _ ⇒ v) a (λ _ ⇒ 0)`
- `differential_of_linear` : `∀ f : U → V, Linear F U V f → ∀ a : U, differential f a f`
- `differential_add` : `∀ (f g : U → V) (a : U) (Lf Lg : U → V), differential f a Lf → differential g a Lg → differential (λ x ⇒ f x + g x) a (λ h ⇒ Lf h + Lg h)`
- `differential_smul` : `∀ (c : F) (f : U → V) (a : U) (Lf : U → V), differential f a Lf → differential (λ x ⇒ c *: f x) a (λ h ⇒ c *: Lf h)`

**ChainRule** (U V W)
ops: `differential : (U → V) → U → (U → V) → Prop`, `differential : (V → W) → V → (V → W) → Prop`, `differential : (U → W) → U → (U → W) → Prop`
- `differential_comp` : `∀ (f : U → V) (g : V → W) (a : U) (Lf : U → V) (Lg : V → W), differential f a Lf → differential g (f a) Lg → differential (λ x ⇒ g (f x)) a (λ h ⇒ Lg (Lf h))`

**TableSpace** (F I T) over `VectorSpace F T`
- `get_zero` : `∀ i : I, get 0 i == 0`
- `get_add` : `∀ (s t : T) (i : I), get (s + t) i == get s i + get t i`
- `get_smul` : `∀ (a : F) (t : T) (i : I), get (a *: t) i == a * get t i`

**Tree** (A Tr)
ops: `tip : A → Tr`, `fork : Tr → Tr → Tr`
- `equivalence` : `Equivalence (@equ Tr equ)`
- `tip_proper` : `Proper (equ ==> equ) (tip : A → Tr)`
- `fork_proper` : `Proper (equ ==> equ ==> equ) (fork : Tr → Tr → Tr)`
- `tip_injective` : `∀ a b : A, lit a == lit b → a == b`
- `fork_injective` : `∀ t u t' u' : Tr, fork t u == fork t' u' → t == t' /\ u == u'`
- `tip_not_fork` : `∀ (a : A) (t u : Tr), tip a =/= fork t u`
- `tree_induction` : `∀ P : Tr → Prop, Proper (equ ==> iff) P → (∀ a : A, P (tip a)) → (∀ t u : Tr, P t → P u → P (fork t u)) → ∀ t : Tr, P t`

**StackMachine** (V C S) extends `Monoid`
ops: `binop : V → V → V`, `push_instr : V → C`, `op_instr : C`, `exec : C → S → S`, `push : V → S → S`
- `values` : `Equivalence (@equ V equ)`
- `stacks` : `Equivalence (@equ S equ)`
- `binop_proper` : `Proper (equ ==> equ ==> equ) binop`
- `exec_proper` : `Proper (equ ==> equ ==> equ) exec`
- `push_proper` : `Proper (equ ==> equ ==> equ) push`
- `exec_empty` : `∀ s : S, exec 0 s == s`
- `exec_antiadditive` : `∀ x y : C, exec (x + y) == (λ s : S ⇒ exec y (exec x s))`
- `exec_push` : `∀ (v : V) (s : S), exec (push_instr v) s == push v s`
- `exec_op` : `∀ (a b : V) (s : S), exec op_instr (push b (push a s)) == push (binop a b) s`

**Compiler** (V E C S) extends `StackMachine`, `Tree`
ops: `tip : V → E`, `fork : E → E → E`, `binop : V → V → V`, `push_instr : V → C`, `op_instr : C`, `exec : C → S → S`, `push : V → S → S`, `evaluate : E → V`, `compile : E → C`
- `evaluate_proper` : `Proper (equ ==> equ) evaluate`
- `compile_proper` : `Proper (equ ==> equ) compile`
- `evaluate_lit` : `∀ v : V, evaluate (lit v) == v`
- `evaluate_node` : `∀ e f : E, evaluate (node e f) == binop (evaluate e) (evaluate f)`
- `compile_lit` : `∀ v : V, compile (lit v) == push_instr v`
- `compile_node` : `∀ e f : E, compile (node e f) == compile e + (compile f + op_instr)`

**Dictionary** (K V D)
ops: `empty : D`, `insert : K → V → D → D`, `find : D → K → option V`
- `entries` : `Equivalence (@equ D equ)`
- `insert_proper` : `Proper (equ ==> equ ==> equ ==> equ) (insert : K → V → D → D)`
- `find_congruent` : `∀ (d : D) (k k' : K), k == k' → find d k = find d k'`
- `find_empty` : `∀ k : K, find (empty : D) k = None`
- `find_insert_here` : `∀ (k k' : K) (v : V) (d : D), k == k' → exists v' : V, find (insert k v d) k' = Some v' /\ v' == v`
- `find_insert_there` : `∀ (k k' : K) (v : V) (d : D), k =/= k' → find (insert k v d) k' = find d k'`

**Lexicon** (C N)
ops: `space : C → Prop`, `digit_value : C → N`, `backslash dot_c lparen rparen eqsign semi colon dash gt : C`
- `characters` : `Equivalence (@equ C equ)`
- `space_proper` : `Proper (equ ==> iff) space`
- `alpha_proper` : `Proper (equ ==> iff) alpha`
- `digit_proper` : `Proper (equ ==> iff) digit`
- `digit_value_proper` : `Proper (equ ==> equ) digit_value`
- `space_dec` : `∀ c : C, Decidable (space c)`
- `alpha_dec` : `∀ c : C, Decidable (alpha c)`
- `digit_dec` : `∀ c : C, Decidable (digit c)`

**Streaming** (C S)
ops: `next : S → option (C * S)`

- next_decides_step : ∀ (s : S) (c : C) (s' : S), step s c s' ↔ exists c0 : C, next s = Some (c0, s') /\ c0 == c

**List** (O K) over Bicartesian O extends InitialAlgebra
ops: build : shape den x L ~> L, cata : ∀ x0 : O, shape den x x0 ~> x0 → L ~> x0, den : K → O, x : K, L : O

**Either** (O K) over Bicartesian O extends InitialAlgebra
ops: build : shape den a b E ~> E, cata : ∀ x : O, shape den a b x ~> x → E ~> x, den : K → O, a b : K, E : O

**Pair** (O K) over Bicartesian O extends InitialAlgebra
ops: build : shape den a b P ~> P, cata : ∀ x : O, shape den a b x ~> x → P ~> x, den : K → O, a b : K, P : O

**NaturalNumbers** (O) over Bicartesian O extends InitialAlgebra
ops: build : shape O N ~> N, cata : ∀ x : O, shape O x ~> x → N ~> x, N : O

**Tree** (O K) over Bicartesian O extends InitialAlgebra
ops: build : shape den x T ~> T, cata : ∀ x0 : O, shape den x x0 ~> x0 → T ~> x0, den : K → O, x : K, T : O

**LambdaTerm** (O K) over Bicartesian O extends InitialAlgebra
ops: build : shape den n T ~> T, cata : ∀ x : O, shape den n x ~> x → T ~> x, den : K → O, n : K, T : O